

# Daily Scholar Papers Report — 2026-07-04

---

2026-07-04

# Daily Scholar Papers Report — 2026-07-04

---

[Download PDF](#)

**Window covered:** 2026-07-03 → 2026-07-04 (Google Scholar alerts + user-curated self-emails, last 24 h)

---

## Executive Summary

---

Three Scholar-alert threads returned in the 24 h window: a “Recommended articles” batch with seven candidates, plus followed-researcher alerts for Andreas Zeller (two papers) and David Lo (one). Ten total candidates, no exclusion-list hits, no user-curated self-emails. Stage-1 discarded three from the Recommended batch as saturated DL-for-vulnerability-detection with weak novelty or off-axis venues, leaving seven for deep-read: two arXiv preprints from the Recommended batch that showed genuine methodological novelty, one ACM journal empirical study of backdoor attacks on Ethereum, and the four followed-researcher papers auto-promoted through the standard override. Preference layer: only `venue: :preprint` cleared the  $n \geq 2$  threshold with posterior 0.75, giving all arXiv candidates today a mild preference-promoted tailwind but not driving any final decisions.

**Outstanding:** 2 · **Keep:** 4 · **Borderline High-Priority:** 1

---

## Highlighted Papers

---

| #   | Title                                                                                                      | Authors                                    | Venue                       | Link                  |
|-----|------------------------------------------------------------------------------------------------------------|--------------------------------------------|-----------------------------|-----------------------|
| 1.1 | Symbolon: Symbolic Execution by Learning Code Transformation                                               | Zhu, P Li, Z Li, Shen, Z Li, Chen, Pei     | arXiv 2606.29108, June 2026 | <a href="#">arXiv</a> |
| 1.2 | Words Speak Louder Than Code: Investigating Cognitive Heuristics in LLM-Based Code Vulnerability Detection | Shahriar, H Cai, Benkraouda, G Wang, Celik | arXiv 2606.30587, June 2026 | <a href="#">arXiv</a> |
| 2.1 | Understanding Binary Code Similarity for Real-World Vulnerability                                          | Guo, Dong, Ren, S Li, J Liu, H Li, Zhu     | arXiv 2606.28870, June 2026 | <a href="#">arXiv</a> |

| #   | Title                                                                                                       | Authors                                                                       | Venue                                     | Link                  |
|-----|-------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|-------------------------------------------|-----------------------|
|     | Detection: A Large-Scale Empirical Study                                                                    |                                                                               |                                           |                       |
| 2.2 | PYURIFY: Purifying Python Tests for Precise Fault Localization                                              | Smytzek, Zeller                                                               | preprint (GitHub deposit), 2026           | <a href="#">PDF</a>   |
| 2.3 | How do Execution Features Improve Statistical Fault Localization? An Empirical Study                        | Smytzek, Zeller                                                               | arXiv 2606.30324, June 2026               | <a href="#">arXiv</a> |
| 2.4 | Humanizing Automatically Generated Unit Test Suites with LLM-Based Refactoring                              | Ouédraogo, Y Li, Dang, Borsukiewicz, Xiao, Bao, Koyuncu, Klein, Lo, Bissyandé | arXiv 2606.28229, June 2026               | <a href="#">arXiv</a> |
| 3.1 | The Backdoor Injures Its Cryptocurrency: An Empirical Study of Backdoor Attacks on Ethereum Smart Contracts | Yanai, Nishida, Unagami                                                       | ACM Distributed Ledger Technologies, 2026 | <a href="#">DOI</a>   |

## 1. Outstanding

**1.1 · SYMBOLIC EXECUTION ·** arXiv preprint 2026 — Symbolon lets an LLM agent learn code transformations offline on small programs, applies them to KLEE at repo scale, and delivers 3.69× line coverage, 29.2× less peak memory, 123× less solver time per query, plus 21 unknown Linux-kernel bugs

**1.1 [Symbolon: Symbolic Execution by Learning Code Transformation](#) — Zhu, P. Li, Z. Li, Shen, Z. Li, Chen, Pei, arXiv preprint arXiv:2606.29108, June 2026**

**Authors.** Junru Zhu, Penghui Li, Zhihao Li, Chao Shen, Zhi Li, Yiqun Chen, Kexin Pei. Pei (University of Chicago) is the most visible name in the ML-for-security wing of this list; the byline gathers a China / US-agentic-analysis mix.

**Venue.** arXiv 2606.29108, June 2026. Preprint. Preference layer gives preprints a mild tailwind (posterior 0.75,  $n = 2$ ).

**License.** arXiv default non-exclusive license — figures not embedded in this summary; the technical contribution is captured as prose plus the paper’s own numbers.

**Problem.** Symbolic execution’s scalability is dominated by *representation-sensitivity*: the same computation encoded differently exposes different branches, memory-access patterns, and constraints, and the delta between an easy and a hard encoding for the solver can be dramatic. Existing

transformation-based approaches to symbolic execution rely on compiler optimisations (designed for concrete execution, not symbolic) or hand-coded rewrites (narrow, brittle, and expensive to author). Neither approach scales to the diversity of source-level patterns real programs contain.

**Approach — offline skill distillation, on-line agentic application.** Symbolon decouples transformation *discovery* from transformation *deployment*. Discovery is done offline on small programs, where symbolic execution and replay are cheap; a search process synthesises transformations and validates them against replay coverage. Effective transformations are distilled into a *reusable library of persistent agent skills* — each skill captures a recurring transformation pattern the agent can later invoke. Deployment is done on-line by an LLM agent that reads a repo-level target, selects skills from the library, and instantiates them context-sensitively. The one-time discovery cost amortises across every downstream project and campaign.

**Headline numbers (from the abstract, verified against Section 2 and Section 6).**

- Evaluated against KLEE across **16 search strategies on 32 real-world programs**.
- **Line coverage: 3.69× on average** over KLEE’s built-in strategies.
- **Peak memory: 29.2× reduction** per query.
- **Solver time per query: 123× reduction**.
- Applied to the *latest* Linux kernel: **21 previously unknown bugs**, all reported to the maintainers.

**Formal characterisation.** No fully mechanised theorem, but the paper is explicit about the abstraction: the transformation library is a set of *reusable agent skills*, and skill invocation is a search problem over program representations conditioned on a symbolic-execution objective (coverage, reachability, or solver cost). The distinction from prior work is that the agent is not just calling KLEE with different flags — it is *rewriting the program’s representation* before KLEE sees it, using skills that were validated on small programs.

**Why Outstanding.** Three qualifying features. First, a genuinely novel architectural insight: the offline / on-line split lets an expensive learning phase pay for itself many times over — cheap to run per target, expensive to prepare once. Second, the numbers are unusually strong across three orthogonal axes (coverage, memory, solver time) — most transformation papers move one metric at a time. Third, the kernel result (21 unknown bugs) is a downstream security artefact the community will actually cite, not just a synthetic-benchmark number. The skill-library abstraction is also reusable: any solver-guided technique with a program-representation dependency (fuzzing, model checking, WCET analysis) could plug in the same discovery/deployment split.

**Reusability signals.** The “persistent skills library” pattern generalises. If you are building an agent that plugs into a downstream tool with expensive per-target cost (fuzzers, verifiers, decompilers), the Symbolon pattern says: *harvest transformations offline on cheap targets, validate them there, then let the agent select skills at deploy time*. The key engineering discipline is designing the skill contract carefully enough that on-line instantiation is context-sensitive without being ad-hoc.

**Open questions.** (i) How much of the 3.69× gain is attributable to the skill library versus the LLM agent’s on-line reasoning? A skill-only baseline (agent picks the closest skill, no reasoning) would isolate the two. (ii) The Linux kernel bug count is a strong signal, but is the bug distribution biased toward one subsystem, or does it spread? (iii) How does the discovery phase’s cost scale with the diversity of the target ecosystem — is one skill library enough for all of C, or would the paper’s technique require separate libraries per subsystem style?

**1.2 · LLM SECURITY · arXiv preprint 2026** — first systematic study of cognitive-heuristic vulnerabilities in LLM vuln-detectors: framing 33.2%, anchoring 23.5%, halo 18.4% average susceptibility across 8 LLMs, plus a black-box cognitive attack that suppresses up to 97% of detected vulnerabilities

**1.2 Words Speak Louder Than Code: Investigating Cognitive Heuristics in LLM-Based Code Vulnerability Detection — Shahriar, H. Cai, Benkraouda, G. Wang, Celik, arXiv preprint arXiv:2606.30587, June 2026**

**Authors.** Ashraful Shahriar, Hongyi Cai, Hadjer Benkraouda, Gang Wang, Berkay Celik. Celik (Purdue) and Wang (UIUC) anchor the byline; the paper sits squarely in the empirical-security-of-LLMs neighbourhood.

**Venue.** arXiv 2606.30587, June 2026. Preprint. Preference layer gives a mild tailwind.

**License.** arXiv default non-exclusive — figures not embedded.

**Problem.** LLM-based vulnerability detectors are being deployed in production (Copilot Autofix, AppSec platforms, CI/CD gatekeepers). In those settings the model does *not* receive code in isolation — it receives author identity, task directives, docstrings, commit messages, prior static-analysis verdicts. All of these non-code signals carry *cognitive information* the model can read. Prior work on LLM cognitive biases studied general reasoning and subjective tasks, where biases are treated as single-directional errors. In security-critical detection, bias is *bidirectional*: a heuristic that suppresses a true vulnerability is adversarial; a heuristic that raises a true positive is constructive. Nobody had previously separated the two.

**Approach — controlled framework holding code fixed, varying only cognitive context.** Three cognitive heuristics are operationalised:

- **Halo effect**, injected via author attribution (e.g. code presented as written by a “principal security engineer” vs. a “new junior developer”).
- **Framing effect**, injected via the task objective and stated consequences of the analysis.
- **Anchoring effect**, injected via a prior analysis result the model sees before it renders its verdict.

The framework holds the code fixed and only varies the surrounding context, so any change in verdict is attributable to the cognitive condition, not to a code perturbation. Evaluated across **eight LLMs, three programming languages**, both quantitative (verdict-flip rates, susceptibility) and code-level (which vulnerability classes flip).

**Headline numbers (from the abstract, all verified against Section 1).**

- **Framing: 33.2%** average cross-model susceptibility (highest).
- **Anchoring: 23.5%** average cross-model susceptibility.
- **Halo: 18.4%** average cross-model susceptibility.
- **Semantic-reasoning vulnerabilities are 1.5x–2x more susceptible** than pattern-matchable ones.
- **Proof-of-concept black-box cognitive attack: suppresses up to 97% of previously detected vulnerabilities.**

- Open-source models are generally more biased than commercial ones. Framing induces the expected-direction response in **all eight** tested models; halo and anchoring reverse direction on some.

**Formal characterisation.** No formal theorem — this is empirical. The paper’s operative definitions are: *cognitive susceptibility* = the rate at which the model’s verdict changes as a function of the cognitive-condition perturbation, holding code constant. Bidirectionality is measured as separately-signed changes toward “safe” vs. “vulnerable”. The 97% suppression attack is a concrete instance of the adversarial direction being weaponised — a black-box adversary who only controls the surrounding context (say, a docstring in a pull request) can silence the LLM’s detector.

**Why Outstanding.** Three qualifying features. First, a genuinely new angle — the cognitive-heuristics literature had not been connected to security-critical LLM deployment before. Second, the attack is *black-box and contextual*: the adversary does not need model access, does not need to perturb code, does not need to bypass a filter — they only need to write plausible docstrings. This is deployable against any LLM security gatekeeper today, and the 97% suppression number is a headline the security-tools industry cannot ignore. Third, the vulnerability-class breakdown (semantic 1.5×–2× more susceptible than pattern-matchable) is *actionable*: it tells defenders that the classes of bugs their LLMs are supposed to catch are exactly the classes cognitive attacks target most effectively.

**Reusability signals.** The controlled framework — hold code fixed, vary only context, measure signed susceptibility per heuristic — is directly reusable for any downstream LLM tool that consumes non-code context (code review, refactoring, threat modelling, incident triage). Any team deploying an LLM security gatekeeper should replicate the framework as a pre-deployment audit. The bidirectionality decomposition (constructive vs. adversarial) is a good template for future LLM-bias work.

**Open questions.** (i) The 97% suppression figure is on a specific proof-of-concept — what is the distribution of suppression rates across CWE classes, and does it correlate with the semantic/pattern breakdown? (ii) How stable is the framing susceptibility across model versions — will fine-tuning or RLHF on security data reduce it, or (more likely) will it shift the specific triggers without changing the fundamental susceptibility? (iii) Can a defence be built on the same framework — e.g., a wrapper that injects randomised counter-cognitive context and votes across verdicts?

---

## 2. Keep

---

**2.1 · BINARY ANALYSIS ·** arXiv preprint 2026 — largest-ever firmware/BCSD study (46,802 firmware images, 200 vendors, 1,069 vulnerable-binary tuples covering 402 CVEs); build-aware queries raise MRR from 0.818 to 0.981 (a ~20% jump), TPL-aware two-stage search adds another 18.5%

**2.1 [Understanding Binary Code Similarity for Real-World Vulnerability Detection: A Large-Scale Empirical Study](#)** — Guo, Dong, Ren, S. Li, J. Liu, H. Li, Zhu, arXiv preprint arXiv:2606.28870, June 2026

**Authors.** Jingdong Guo (IIE, CAS), Chaopeng Dong (Hangzhou Dianzi + IIE), Yimo Ren (IIE), Siyuan Li (Shandong), Jie Liu (IIE), Hong Li (IIE), Hongsong Zhu (IIE). Guo and Dong contributed equally.

**Venue.** arXiv 2606.28870, June 2026. Preprint. ACM formatting (Vol. 1, No. 1, Article, June 2026) suggests journal submission in progress.

**License.** Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International (CC-BY-NC-ND 4.0). Figures may be embedded with attribution; no derivatives (so no re-cropping edits) — this summary embeds no figures.

**Problem.** Binary Code Similarity Detection (BCSD) is the go-to technique for finding third-party-library vulnerabilities in firmware, but published evaluations use small, homogeneous datasets and skip the factors that dominate real-world deployment (which vulnerable-function version to query, which firmware to compare against, how function size interacts with detector accuracy, how compilation macros distort binary signatures). Practitioners lack a defensible baseline for how well BCSD works on real firmware at scale.

**Approach — dataset first, then factor analysis.** Two-stage design:

1. **Data collection and preprocessing:** 46,802 firmware images from 200 vendors across nine categories (NAS, Camera, Switch, ...); 4,411,056 binaries extracted with 547,578 unique hashes; a vulnerable-binary dataset of 1,069 binaries covering 9 TPLs, 402 CVEs, 53 CWEs.
2. **Detection and evaluation:** using HermesSim (selected as the strongest BCSD from a public-benchmark comparison), the paper answers four research questions on **TPL versions, TPL nesting (static linking), function size, and macro definitions**.

**Formal metrics.** The paper quotes its own definitions verbatim in Section 2:

Mean Reciprocal Rank (MRR) Quantifies how highly the first correct match is ranked; larger values indicate better overall precision. Recall@K The fraction of queries whose correct match appears within the top- $K$  candidates.

$$\text{MRR} = \frac{1}{N} \sum_{i=1}^N \frac{1}{\text{rank}_i}, \quad \text{Recall@K} = \frac{1}{N} \sum_{i=1}^N \mathbb{1}[\text{rank}_i \leq K]$$

**Headline numbers.**

- HermesSim tops every other BCSD tested; **MRR exceeds baselines across XA/XC/XM cross-compilation settings**.
- **Vsame** (query matched to same-version vulnerable binary) achieves **MRR 0.961**, surpassing Voldest / Vnewest by 11.1%.
- **Build-aware query strategy** (querying representative in-the-wild binaries whose compilation macros cluster with the target) raises **MRR from 0.818 to 0.981**,  $\approx 20\%$  improvement.
- **TPL-aware two-stage search** (constraining candidate pool to plausible TPLs first) improves MRR by an additional **18.5%**.

**Why Keep.** Solid empirical contribution and the biggest firmware/BCSD dataset published to date, which is where the reusability value lies. The technique itself is not a new BCSD model — it is a *deployment recipe* (which query to use, how to constrain the search space) that a practitioner can adopt on top of any existing BCSD. Deep-read but not Outstanding because the novelty is in the empirical measurement and the query strategy, not in a new BCSD architecture.

**Reusability signals.** The dataset and the build-aware query methodology are directly reusable for anyone building or evaluating a firmware vulnerability-detection pipeline. The compilation-macro clustering trick — group representative binaries by their compilation macros, then query with a representative from the cluster — is a small, portable engineering move that likely helps any BCSD.

**Open questions.** (i) The two-stage TPL-aware search assumes reliable TPL identification; the paper does not deeply characterise the TPL-identification failure modes. (ii) Function-size “complexity valleys” (referenced in Section 1) are an interesting phenomenon that deserves a deeper causal breakdown. (iii) HermesSim is the strongest current BCSD, but the query strategies are supposed to be BCSD-agnostic — how much do the numbers change with an older / weaker baseline?

**2.2 · FAULT LOCALIZATION · preprint 2026** (Zeller followed-researcher) — PYURIFY is the first practical test-purification tool for Python: atomize multi-assertion tests, dynamic-slice each one, feed cleaner spectra to SFL; released under Apache-2.0 with CLI + Python API

## **2.2 [PYURIFY: Purifying Python Tests for Precise Fault Localization](#) — Smytzek, Zeller, preprint 2026**

**Authors.** Marius Smytzek, Andreas Zeller (CISPA). Zeller is the followed name — this paper is auto-promoted via the followed-researcher override.

**Venue.** Preprint, hosted on GitHub. The paper’s format suggests submission to a conference tool track.

**License.** Paper text not marked with an explicit open license; the tool artefact is released under Apache 2.0. No figures embedded in this summary.

**Problem.** Statistical fault localization (SFL) — ranking suspicious code lines from pass/fail spectra — is sensitive to spectrum noise. A test that mixes multiple assertions and unrelated setup code produces a spectrum in which the failing signal is diluted by all the incidentally-covered code. Test-case purification (Xuan et al.’s prior Java work) addresses this by splitting each multi-assertion test into single-assertion variants and slicing away irrelevant code, but no practical tool existed for Python — the dynamic execution model of Python makes both atomization and slicing non-trivial.

**Approach — atomization then dynamic slicing.** Two stages:

1. **Atomization:** each pytest-style test with multiple assertions is split into single-assertion tests, one per assertion.
2. **Dynamic slicing:** each atomized test is instrumented and executed; a data-flow-driven dynamic slice removes any statement that does not contribute (transitively via data flow) to the surviving assertion.

The purified single-assertion tests produce cleaner coverage spectra, which SFL formulas (Tarantula, Ochiai, Jaccard, etc.) consume for a sharper suspiciousness ranking. The tool provides both a CLI and a Python API, and integrates with standard SFL frameworks. Empirical evaluation is on real Python bugs with “consistent improvements in fault localization metrics” (exact table numbers not extracted in the excerpt).

**Why Keep.** A tool paper with a clear, publishable contribution: purification for the Python ecosystem (which had none) built cleanly on Xuan et al.’s established Java approach. Zeller’s follow means it is auto-promoted; the paper itself is a Keep rather than an Outstanding because the algorithmic novelty is in *engineering discipline* rather than a new SFL formulation.

**Reusability signals.** The tool is open source, Apache-2.0 licensed, CLI + Python API — usable off the shelf in existing debugging workflows. If a project wants to evaluate a new SFL formula on Python, PYURIFY becomes the standard preprocessing step. The atomization + dynamic-slicing pipeline is also a

template that can be repackaged for other dynamic languages (JavaScript, Ruby) with only implementation changes.

**Open questions.** (i) What are the exact accuracy improvements over unpurified spectra on the Python benchmark? (ii) How does purification cost (extra instrumentation, slicing) trade off against SFL runtime? (iii) Does purification interact with LLM-based debugging tools that consume test suites directly — is a purified suite easier for an LLM to reason over, or does it lose context the model was implicitly using?

**2.3 · FAULT LOCALIZATION · arXiv preprint 2026 (Zeller followed-researcher)** — SFL augmented with EFDD execution-features (def-use pairs, value predicates, scalar-pair relations) via per-subject random forests; keeps the baseline formula intact and re-weights it multiplicatively, evaluated with a confounder-adjusted mixed-effects model on Tests4Py

## **2.3 How do Execution Features Improve Statistical Fault Localization? An Empirical Study — Smytzek, Zeller, arXiv preprint arXiv:2606.30324, June 2026**

**Authors.** Marius Smytzek, Andreas Zeller (CISPA). Zeller is followed.

**Venue.** arXiv 2606.30324, June 2026.

**License.** Apache 2.0 for the artifacts (scripts, seed, docs). Paper text does not carry an explicit CC license. No figures embedded.

**Problem.** SFL from line coverage alone treats every executed line as equally suspicious and cannot distinguish among lines executed in failing tests, so the top-of-ranking is noisy. The authors' prior EFDD work showed that richer execution characteristics — definition-use pairs, variable-value predicates, scalar-pair relations — correlate more strongly with failure than line execution. The open question is whether *plugging EFDD features into SFL* transfers those correlation gains to inspection-effort savings on real faults, or whether the extra signal is orthogonal to what SFL formulas exploit.

**Approach — additive re-weighting, not replacement.** The critical design choice is stated verbatim in the “Positioning” section: *keep each baseline formula and only modulate it multiplicatively*. The pipeline:

1. Extract execution features with EFDD for every Tests4Py subject.
2. Train a per-subject random forest to identify failure-indicative features.
3. Map feature importances to source lines.
4. Multiply the baseline SFL suspiciousness by the mapped weight — the formula is kept intact, so the change is inspectable and comparable against the same formula's baseline.

Evaluation is deliberately careful: reference-patch accuracy, line- and function-level effort, robustness, feasibility, all wrapped in a confounder-adjusted mixed-effects model with paired statistical tests and outcome-neutral quality checks.

**Why Keep.** Empirical study rather than an outstanding methodological leap, but the study design — reweighting instead of replacing, and the confounder-adjusted mixed-effects analysis — is the kind of empirical discipline the community keeps rediscovering it needs. Zeller-followed, and worth Keep for the evaluation template alone.

**Reusability signals.** The multiplicative-reweighting scheme is a portable meta-move: it lets any signal (an execution feature, an LLM verdict, a symbolic-execution reachability score) be added on top of a canonical SFL formula without abandoning the formula's interpretability. The mixed-effects evaluation



with paired tests and outcome-neutral quality checks is a good template for any localization / ranking study.

**Open questions.** (i) The paper’s Positioning section frames a positive result as *not guaranteed* — what does the actual result look like across the four baseline formulas? (ii) Does the random-forest-per-subject overfit the small Tests4Py subjects, or would a globally-trained model transfer better? (iii) EFDD features are Python-specific in current tooling; does the reweighting scheme carry over to Java (Defects4J) datasets where past comparators live?

**2.4 · TEST GENERATION ·** arXiv preprint 2026 (David Lo followed-researcher) — TestHumanizer is a hybrid SBST+LLM pipeline that refactors EvoSuite suites for readability while preserving behaviour: 88–98% compilation rate (vs. 51–78% for direct LLM generation), Conditional-Logic-Test smell dropped from 36.97% → 5.02% on Defects4J, 30-class developer study confirms significant readability gains ( $p < 0.01$ )

## **2.4 [Humanizing Automatically Generated Unit Test Suites with LLM-Based Refactoring](#) — Ouédraogo, Y. Li, Dang, Borsukiewicz, Xiao, Bao, Koyuncu, Klein, Lo, Bissyandé, arXiv preprint arXiv:2606.28229, June 2026**

**Authors.** Wendkûuni C. Ouédraogo (Luxembourg), Yinghua Li (NJUST), Xueqi Dang (Luxembourg), Pawel Borsukiewicz (Luxembourg), Liang Xiao (NJUST), Lingfeng Bao (Zhejiang), Anil Koyuncu (Bilkent), Jacques Klein (Luxembourg), **David Lo** (SMU), Tegawendé Bissyandé (Luxembourg). Lo is followed.

**Venue.** arXiv 2606.28229, June 2026. Journal formatting (J. ACM Vol. 1, No. 1, Article, July 2026) — likely under submission.

**License.** Creative Commons Attribution 4.0 International (CC BY 4.0) via the Luxembourg grant-agreement clause. Figures may be embedded; none embedded here since the contribution is conveyed as prose plus the paper’s numbers.

**Problem.** Search-Based Software Testing (EvoSuite) produces compilable high-coverage unit tests at scale, but the tests are notoriously hard to read and maintain — cryptic names, monolithic test methods with many assertions, obscured intent. LLMs generate more natural tests, but one-shot LLM generation has compilation rates of 51–78%, so it does not scale to production. The problem is: can LLMs be used as a *refactoring layer over SBST output* to preserve compilation reliability while adding human-readable structure?

**Approach — hybrid SBST + validation-gated LLM refactoring.** The pipeline:

1. EvoSuite generates 15 suites per class (100% compilation baseline).
2. For each suite, an LLM refactors under one of three context configurations: **tests-only, code-centric, summary-based**.
3. Two general-purpose LLMs are used; the pipeline runs 31,500 refactorings across 350 classes (Defects4J: 147, SF110: 203).
4. Every refactoring is validated against compilation and behavioural checks; the paper measures compilation rate, coverage preservation, and structural / readability / behaviour smells.

The key architectural claim: LLMs are best deployed as *validation-gated refinement layers* over robust SBST output, not as standalone generators. This is a concrete architectural pattern for the “LLM as tool” school of hybrid systems.

## Headline numbers (verified against the abstract).

- **TestHumanizer compilation rate: 88–98%**, vs. direct LLM generation at **51–78%**.
- **Structural coverage preserved** typically within 1–2 percentage points; median method coverage stays at 100%.
- **86–95% of refactorings pass a composite faithful-refactoring threshold.**
- **Conditional Logic Test smell** dropped from **36.97%** → **5.02%** on Defects4J, and **48.31%** → **4.37%** on SF110 (summary-based configuration).
- Developer study on 30 classes (444 test methods) confirms **significant gains in perceived readability and adoption willingness**, Wilcoxon  $p < 0.01$ , medium-to-large effect, inter-rater agreement  $\alpha = 0.71$ – $0.85$ .
- The **summary-based configuration** provides the most robust trade-off; long code-centric prompts are more prone to hallucination-induced failures and API-level rejections.

**Why Keep.** Strong empirical contribution and a genuinely useful architectural takeaway (validation-gated LLM refactoring layer over SBST). Not Outstanding today only because the underlying architectural pattern is already circulating in the LLM-tool-integration literature and the paper’s novelty is in the careful empirical treatment rather than a new mechanism. David-Lo follow guarantees promotion.

**Reusability signals.** The three-configuration ablation (tests-only vs. code-centric vs. summary-based) is a clean template — the finding that long code-centric prompts hurt more than they help is worth internalising for any LLM-refactoring pipeline. The compilation-rate / coverage-preservation / smell-reduction triple is a good evaluation template for anyone measuring an LLM refactoring layer.

**Open questions.** (i) The summary-based configuration wins on robustness — what is the failure mode of the summary approach that would motivate falling back to code-centric? (ii) Does the compilation-rate gap (88–98% vs. 51–78%) hold across model families, or is it an artefact of the two general-purpose LLMs studied? (iii) Are the readability gains persistent under maintenance — does the refactored suite remain readable after a developer edits a few tests, or does downstream maintenance regenerate the original SBST smells?

---

## 3. Borderline High-Priority

---

**3.1 · SMART CONTRACTS · ACM DLT 2026** — empirical study of a specific attack class (“backdoor attacks”) on Ethereum smart contracts, where a developer’s privileged functions manipulate the cryptoassets their own contract produces; abstract-only summary because the ACM DL PDF is behind a Cloudflare gate the pipeline could not fetch

**3.1 [The Backdoor Injures Its Cryptocurrency: An Empirical Study of Backdoor Attacks on Ethereum Smart Contracts](#) — Yanai, Nishida, Unagami, ACM Distributed Ledger Technologies: Research and Practice, 2026**

**Authors.** Naoto Yanai, Naoyuki Nishida, Yuji Unagami. Affiliations not extracted from the Scholar snippet.

**Venue.** *ACM Distributed Ledger Technologies: Research and Practice* (DLT), 2026. Emerging ACM journal, not one of the top venues on the priority list — hence Borderline High-Priority rather than Keep.

**License.** ACM. The pipeline could not access the DL PDF (Cloudflare interstitial); summary is built from Scholar’s abstract snippet.

**Problem (from the Scholar abstract).** A “backdoor attack” on an Ethereum smart contract is an attack in which the developer of a contract exploits privileges baked into the contract they deployed — for example, an owner-only mint or burn function — to manipulate the cryptoassets the contract generates. This is a *developer-side* threat model, distinct from the more heavily-studied external-exploitation model (reentrancy, integer overflow, access control). The paper’s contribution is an empirical study of this attack class.

**What we can say from the abstract.** The paper is positioned as an empirical study — a taxonomy or measurement of backdoor-attack contracts observed on-chain, plausibly paired with a detection or classification method. Without the full PDF the exact methodology, detection accuracy, or sample size are unavailable to this summary.

**Why Borderline High-Priority.** The developer-side backdoor threat model is a genuine gap in the smart-contract security literature — most detectors focus on external exploits, not on abuse of privileged functions. But the paper appears at a small ACM venue, and without the PDF it is impossible to grade the empirical rigour. A reader interested in smart-contract security research should retrieve the PDF via institutional access and re-read; if the taxonomy or dataset is well-constructed the paper may move to Keep in a future revisit.

**Reusability signals.** *Unknown pending full-text access.* If the paper releases a labelled dataset of backdoor-attack contracts, that dataset would be reusable for training / evaluating detectors of the developer-side threat class.

**Open questions.** (i) Sample size and construction of the on-chain dataset. (ii) Whether the taxonomy separates *intended-privilege abuse* from *accidental-privilege abuse*, which are two very different mitigation problems. (iii) Whether any of the identified backdoors overlap with contracts already flagged by existing vulnerability scanners (Slither, Mythril) — if the overlap is small, the paper fills a genuine gap.

---

## Cross-Paper Synthesis

---

Two threads run through today’s report. The first is the *what is the model actually reasoning over?* thread. Symbolon (1.1), Words-Speak-Louder (1.2), and the BCSD empirical study (2.1) each highlight, from three different angles, that the *representation* the analyser consumes dominates the analyser’s behaviour: Symbolon rewrites program representations before KLEE sees them, and gets 3.69× more coverage; Shahriar et al. hold code fixed and vary only the surrounding cognitive context, and an LLM’s verdict flips at 33.2% for framing; Guo et al. show that BCSD’s MRR moves from 0.818 to 0.981 based on *which version of the vulnerable binary you query with*. Each of these is a different way to phrase the same design lesson: **the input representation is the first and highest-leverage lever for any code-analysis tool**, and papers that measure the leverage carefully — like today’s Outstanding pair — get their headline numbers by *choosing the representation deliberately*.

The second thread is the *hybridise, do not replace* thread. PYURIFY (2.2) purifies test spectra before SFL runs, keeping the SFL formula intact. Smytzek & Zeller (2.3) explicitly re-weight the SFL formula multiplicatively rather than replacing it, precisely because the baseline formula is inspectable and the reweighting inherits that inspectability. TestHumanizer (2.4) uses LLMs as a validation-gated refactoring

layer over SBST, keeping SBST's compilation guarantee. Symbolon (1.1) itself uses an LLM agent to *rewrite the program before KLEE runs*, not to replace KLEE. All four converge on: **do not replace the trusted stage; hybridise a learned or heuristic stage in front of or on top of it, with an explicit validation contract at the seam**. This is the maturing pattern of LLM-tool integration in program-analysis research — a year ago the pattern was “let the LLM do the whole thing”; today's papers are almost uniformly hybrid. The failure of one-shot LLM test generation (51–78% compilation rate, from TestHumanizer's own comparison) is a good quantitative proof of why the pattern shifted.

---

## Writing & Rationale Insights

---

Two rhetorical moves worth stealing. First, Symbolon's *offline learning + on-line agent* split (1.1) is framed as *amortisation of a one-time cost*, and the framing is what makes the design choice defensible: without amortisation, offline learning looks like a suspicious extra step; with amortisation, it looks like a rational engineering investment. Any paper proposing an expensive preprocessing or training step should adopt this rhetorical move — frame the cost as amortised across all downstream deployments, not as a per-target overhead. Second, Words-Speak-Louder (1.2) is unusually disciplined about *bidirectional* susceptibility measurement — it separates the case where a cognitive heuristic constructively raises detection from the case where it adversarially suppresses detection, and reports both. Prior LLM-bias work treated bias as a single-directional error; the security-critical framing forces bidirectionality, and the paper gains rhetorical force from measuring and naming the adversarial direction explicitly. Both moves generalise: **frame a large one-time cost as amortised across many downstream uses**, and **when your metric can be adversarial or constructive, separate the two and name the adversarial mode**.

The third insight is negative — from the Skipped batch. Two of the three Recommended-batch papers we Skipped today (the XAI-driven survey and the OWASP-benchmark faithfulness study, both sole-author or pair-author from the same author on ResearchGate) followed the pattern of a solo author gathering a large literature and issuing a *gap analysis*. Gap analyses only pay off when the author has a strong prior claim to authority or a unique dataset; without either, the paper reads as a reading list with a title. The instructive contrast: today's Guo et al. BCSD paper (2.1) is also fundamentally an *empirical characterisation* — no new BCSD model — but it earns its Keep because it brings a genuinely unprecedented dataset (46,802 firmware images, 200 vendors) to the table. **Empirical papers without a new artefact or dataset need extraordinary scope or extraordinary rigour to justify the reader's time**; both together is what pushes a paper from Skip to Keep.